NEXTPLAYG · ASSESSMENT

Task Manager

Solution overview, data model, setup and delivery notes

Prepared by: Mark Anthony Calla Vargaya

01 / SOLUTION

A private, lightweight task board

NextPlayG is a responsive web task manager organized into To Do, In Progress, In Review and Done. Users can create, rename, move and delete tasks, and undo the most recent deletion. Tasks can be moved by drag and drop or through a status selector.

The frontend uses React, TypeScript and Vite. Supabase Auth supplies anonymous guest sessions, while Postgres stores the board. The interface uses a sports-inspired palette with Poppins for controls and Playfair Display for the main heading.

DELIVERY LINKS

Live frontend	Pending: live frontend URL not provided
GitHub repository	Pending: GitHub repository URL not provided

DESIGN DECISIONS

- **Private by default:** tasks and labels belong to the current guest; database row-level security scopes operations to the authenticated user ID.
- **Low-friction entry:** the app reuses an anonymous session and does not require account registration.
- **Data preservation:** existing browser-local tasks are imported once using deterministic IDs; the original local backup is retained.
- **Responsive operation:** columns reflow across screen sizes; controls have labels, keyboard focus states and a non-drag alternative.

Guest sessions are tied to browser data. There is no account recovery or cross-device synchronization; assignee names are private notes and do not grant access.

02 / DATA MODEL

Complete database schema

PostgreSQL in Supabase. UUIDs use `gen_random_uuid()`; `pgcrypto` is enabled. The complete SQL is in `supabase.sql`.

public.tasks

Column	Type	Constraints / purpose
`id`	uuid	Primary key; generated UUID by default.
`title`	text	Required; trimmed length must be greater than zero.
`description`	text	Required; defaults to an empty string.
`status`	text	Required; default `todo`; one of `todo`, `in_progress`, `in_review`, done.
`priority`	text	Required; default `normal`; one of `low`, `normal`, `high`.
`due_date`	date	Optional due date.
`user_id`	uuid	Required foreign key to `auth.users(id)`; cascades on user deletion.
`created_at`	timestamptz	Required; defaults to `now()`.
`assignee`	text	Required; defaults to an empty string; informational name only.
`labels`	text[]	Required; defaults to an empty array; stores the task's label names.

public.labels

Column	Type	Constraints / purpose
`id`	uuid	Primary key; generated UUID by default.
`user_id`	uuid	Required foreign key to `auth.users(id)`; cascades on user deletion.
`name`	text	Required; trimmed length 1–40 characters.
`created_at`	timestamptz	Required; defaults to `now()`.

Indexes: `tasks(user_id, status)` and a unique index on `labels(user_id, lower(trim(name)))`.

Security: row-level security is enabled for both tables. Select, insert, update and delete policies require `auth.uid() = user_id`; update also checks the resulting owner. The authenticated role receives label-table privileges subject to RLS.

Task labels are stored as a text array rather than foreign keys to the catalog; the current interface creates and assigns labels but does not rename or remove catalog entries.

03 / RUNNING AND EVALUATING

Setup and features

Run locally

1. Use Node.js 24 (the project was tested on 24.13.1) and npm. Open a terminal in `nextplayg-task-board`.
2. Install dependencies with `npm install`.
3. Create a Supabase project, enable anonymous sign-ins, and execute `supabase.sql` in its SQL Editor. For an existing earlier schema, execute `supabase-features-migration.sql`.
4. Create a local `.env` containing the project's URL and publishable key. Do not use a service-role key in frontend code.

```
VITE_SUPABASE_URL=<your-project-url>
VITE_SUPABASE_PUBLISHABLE_KEY=<your-publishable-key>
npm run dev
```

Open the local URL printed by Vite. The `.env` file is excluded from version control. Restart the dev server after changing environment variables.

Advanced features

- **Custom labels:** create a private label catalog and assign multiple labels while editing a task.
- **Due-date indicators:** cards mark overdue items, items due today and items due within three days; completed tasks are not flagged overdue.
- **Search and filters:** title search combines with priority, assignee and label filters. Filtering runs on the loaded tasks in the browser.
- **Priority and assignee:** tasks support low, normal and high priority and an optional assignee name; the name is informational, not a shared account.

Tradeoffs and next steps

- Anonymous sessions remove sign-up friction but cannot be recovered after browser data is cleared or on another device. Add optional account linking and recovery if cross-device access becomes a requirement.
- No real-time updates or collaboration are implemented. Consider subscriptions, conflict handling and a permissions model for shared boards.
- Labels are text arrays, so catalog renames or deletions would require coordination. A normalized join table would strengthen referential integrity.
- Expand automated browser, accessibility, migration and RLS integration coverage, and provide a deliberate offline/conflict strategy.

Validation: production build and lint passed; 11 local tests cover filters, due dates and storage/import behavior. Task CRUD and task ownership were tested against Supabase; advanced UI behavior was tested with simulated responses. The expanded table and columns responded to read-only schema checks; end-to-end label RLS has not been verified against the live database.